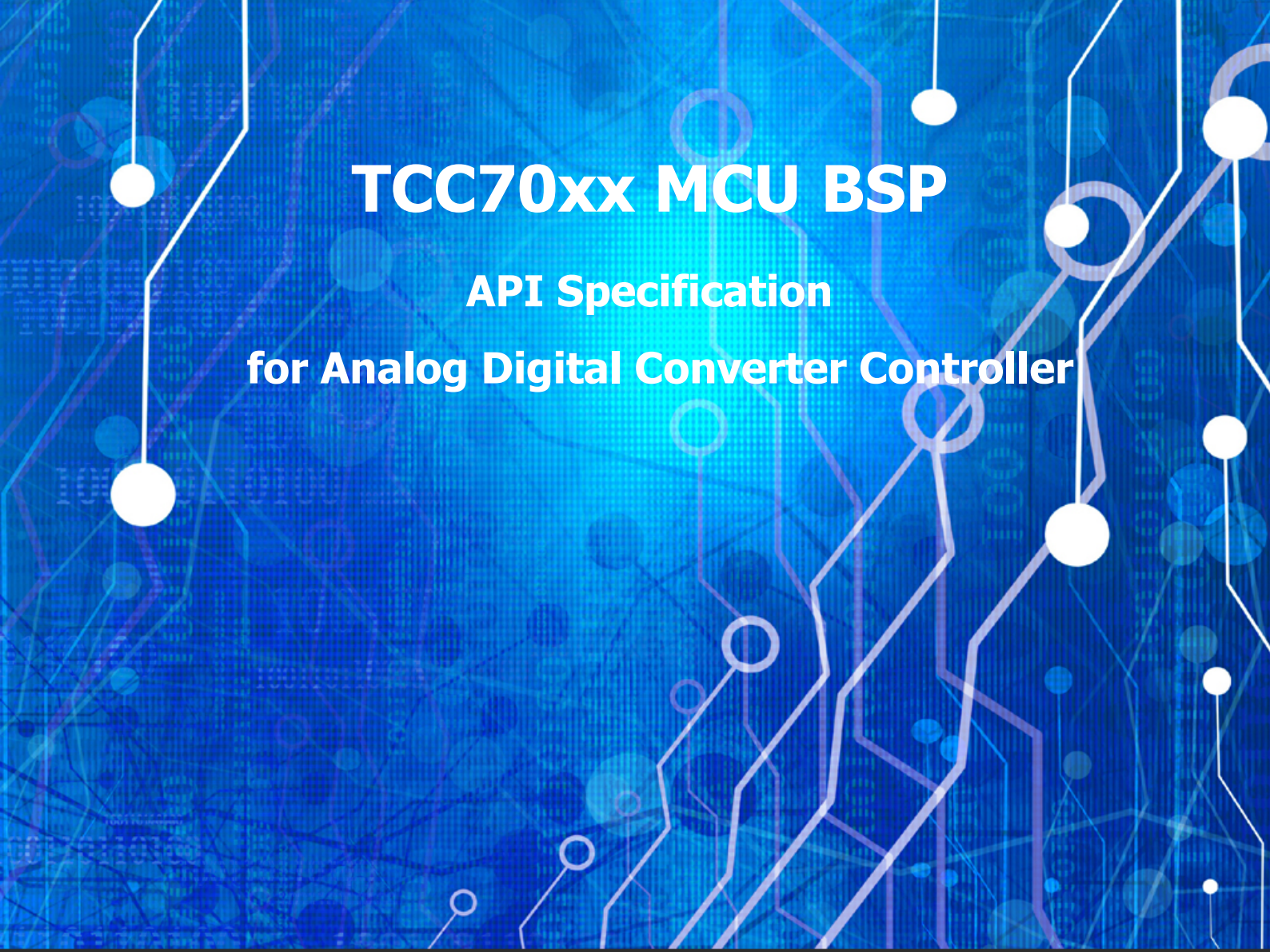




TCC70xx MCU BSP

API Specification

for Analog Digital Converter Controller

Rev. 0.10 [A]

2021-09-03

Preliminary version

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit www.telechips.com for more information.

© 2021 Telechips Inc. All rights reserved.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files.....	4
4	Enumerations	5
4.1	ADCCChannel_t.....	5
4.2	ADCMode_t.....	6
5	Constants.....	7
5.1	ADC Module	7
5.2	Configurations	7
5.3	Error Numbers.....	8
6	APIs	9
6.1	ADC_Init.....	9
6.2	ADC_Read	10
6.3	ADC_AutoScan	11
7	How to use ADC Driver.....	12
7.1	Initialize ADC Driver.....	12
7.2	Read Converted Value	12
8	References	13
9	Revision History	14
	Rev. 0.10: 2021-09-03	14

Figures

Figure 3.1	Location of Source Files	4
------------	--------------------------------	---

Tables

Table 2.1	Terms and Abbreviation	3
-----------	------------------------------	---

1 INTRODUCTION

This document describes the analog digital converter controller device driver (hereinafter referred to as ADC driver) of TCC70xx MCU BSP. For more detailed information on the analog digital converter controller in the TCC70xx MCU Sub-system, refer to Chapter 15 Analog Digital Converter (ADC) Controller in “*TCC70xx Full Specification*” [1].

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

ADC	Analog Digital Converter
------------	--------------------------

3 SOURCE FILES

3.1 Location of Source Files

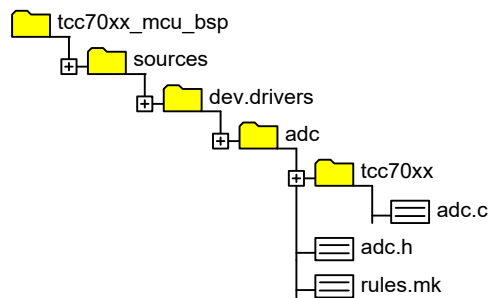


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `adc.c`
 - Source of ADC driver
- `adc.h`
 - API header of ADC driver
- `rules.mk`
 - Rules used for build

4 ENUMERATIONS

4.1 ADCChannel_t

Enumeration variables for ADC channel value of ADC user interface API. TCC70xx ADC supports 12 channels.

Declaration

```
typedef enum __ADC_CHANNEL__  
{  
    ADC_CHANNEL_0      = 0,  
    ADC_CHANNEL_1      = 1,  
    ADC_CHANNEL_2      = 2,  
    ADC_CHANNEL_3      = 3,  
    ADC_CHANNEL_4      = 4,  
    ADC_CHANNEL_5      = 5,  
    ADC_CHANNEL_6      = 6,  
    ADC_CHANNEL_7      = 7,  
    ADC_CHANNEL_8      = 8,  
    ADC_CHANNEL_9      = 9,  
    ADC_CHANNEL_10     = 10,  
    ADC_CHANNEL_11     = 11,  
    ADC_CHANNEL_MAX     = 12  
} ADCChannel_t;
```

Description

Value	Description
ADC_CHANNEL_xx	Channel number value for each ADC channel
ADC_CHANNEL_MAX	The maximum number of channels

Remark

None

4.2 ADCMode_t

Enumeration variables for selecting ADC mode

Declaration

```
typedef enum __ADC_MODE__  
{  
    ADC_MODE_NORMAL    = 0,  
    ADC_MODE_IRQ       = 1,  
    ADC_MODE_DMA       = 2  
} ADCMode_t;
```

Description

Value	Description
ADC_MODE_NORMAL	The conversion value can be read in the ADC sampled data register (SD_OUT0 to SD_OUT11).
ADC_MODE_IRQ	IRQ interrupt is generated when the conversion operation is completed.
ADC_MODE_DMA	The conversion value can be read in the designated location of the DMA.

Remark

None

5 CONSTANTS

5.1 ADC Module

Constant values for TCC70xx ADC Hardware Module.

Declaration

```
#define ADC_MODULE_0 (0)
#define ADC_MODULE_1 (1)
```

Description

Value	Description
ADC_MODULE_0	ADC0 Physical Device Number
ADC_MODULE_1	ADC1 Physical Device Number

5.2 Configurations

Constant values for configuration value for API

Declaration

```
#define ADC_CONV_TIMEOUT (2UL)
```

Description

Value	Description
ADC_CONV_TIMEOUT	Timeout for Done bit. It waits to set done bit to 1 during $ADC_CONV_TIMEOUT * ADC_CONV_CYCLE$ cycles in <i>ADC_Read</i> API.

5.3 Error Numbers

Constant values for error numbers

Declaration

```
#define ADC_ERR_INVALID_MODULE    (-1)
```

Description

<i>Value</i>	<i>Description</i>
<code>ADC_ERR_INVALID_MODULE</code>	<i>Invalid module number is passed into <code>ADC_Init</code> API.</i>

6 APIs

6.1 ADC_Init

Initializes the ADC. Regardless of the input type, the *ADC_Init()* function performs ADC clock enable.

There are three types of modes in ADC as follows:

- Normal mode: Default mode
- IRQ mode: When conversion is completed, this mode can generate an interrupt.
- DMA mode: By using DMA, this mode can read the conversion result.

Declaration

```
void ADC_Init
(
    uint8  Type,
    uint8  ucModule
);
```

Parameters

Value	Description
Type	[in] ADC Mode (Normal, IRQ, DMA). Refer to ADCMode_t .
ucModule	[in] ADC Module number. 0 for ADC0, 1 for ADC1

Return

None

Remark

None

6.2 ADC_Read

Starts the ADC conversion using the *ADC_SMP_CMD* register for single channel, checks the conversion done bit, and reads the conversion result value from the *SDOUT* register. This function does not support thread safe and reentrancy on the same channel.

Declaration

```
uint32 ADC_Read
(
    ADCChannel_t    channel,
    uint8           ucModule,
    int32           siWatch
);
```

Parameters

Value	Description
channel	[in] ADC Channel number. Refer to ADCChannel_t .
ucModule	[in] ADC Module Number. 0 for ADC0, 1 for ADC1.
siWatch	[in] Enable monitoring mode for debugging. It must be 0 by default.

Return

Value	Description
uint32	ADC conversion value, conversion value range is 0 to 4096 (12 bits).

Remark

None

6.3 ADC_AutoScan

Calls **ADC_Read()** function internally to convert input analog signal for all ADC channels and to read the conversion values one by one. This function does not implement consecutive conversion and not support thread safe and reentrancy. This function must be used in only one task of the all tasks such as user tasks and system tasks.

Declaration

```
uint32 * ADC_AutoScan
(
    uint8  ucModule,
    int32  siWatch
);
```

Parameters

Value	Description
ucModule	[in] ADC Module Number. 0 for ADC0, 1 for ADC1.
siWatch	[in] Enable monitoring mode for debugging. It must be 0 by default.

Return

Value	Description
uint32 *	Static array address storing all channel conversion values

Remark

None

7 HOW TO USE ADC DRIVER

7.1 Initialize ADC Driver

```
void BSP_Init()
{
    ...
    ADC_Init(ADC_MODE_NORMAL, ADC_MODULE_0); //ADC Normal Mode for ADC0 initialization
    ADC_Init(ADC_MODE_NORMAL, ADC_MODULE_1); //ADC Normal Mode for ADC1 initialization
    ...
}
```

Note: Refer to sources/dev.driver/common/tcc70xx/bsp.c

7.2 Read Converted Value

```
uint32 * ADC_AutoScan
(
    uint8          ucModule,
    int32          siWatCh
)
{
    uint32 ch;
    uint32 read_data;
    static uint32 adc_scan_val[ADC_CHANNEL_MAX];

    read_data = 0;

    for (ch = 0 ; ch < (uint32)ADC_CHANNEL_MAX ; ch++) // read every channels for ADC module
    {
        read_data      = ADC_Read((ADCChannel_t)ch, ucModule, siWatCh);
        adc_scan_val[ch] = read_data;

        if ((siWatCh >= ADC_CHANNEL_0) && (siWatCh < ADC_CHANNEL_MAX))
        {
            SAL_TaskSleep(100);
        }
    }

    return adc_scan_val;
}
```

Note: Refer to sources/dev.driver/adc/tcc70xx/adc.c

8 REFERENCES

- [1] TCC70xx Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

9 REVISION HISTORY

Rev. 0.10: 2021-09-03

- Preliminary version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."